GIS算法设计与分析

第3次课 · 路网模型与拓扑构建

从原始数据到可导航网络——G2组核心任务

图论建模 · 节点合并 · 边拆分 · 拓扑构建 · 转向限制

授课对象：大三本科生 | 授课时长：2小时

上次课回顾与本课定位

第2课回顾

地图数据生产：OSM格式、数据清洗12类问题、质量评估5维度与多源融合。G1组输出标准化CSV

本课位置

G2组（路网模型组）核心任务课。G1→G2→G3，承上启下：消费G1的清洗数据，为G3空间索引和G4路径规划提供路网

核心问题

如何从OSM原始点线数据构建可用于路径规划的道路网络？节点合并/边拆分/拓扑构建/转向限制

时间分配

回顾、核心讲解、课堂练习与总结

学习目标

学完本次课，你应该能回答这四个问题

01



道路网络的图论建模方法是什么？为什么要用有向加权图？节点/边/权重分别代表什么？

02



从OSM数据到路网模型的转换流程是什么？节点合并/边拆分/属性计算/拓扑构建，每步解决什么问题？

03



拓扑关系的三种表达（邻接表/前向星/边表）各有什么优缺点？适用什么场景？

04



转向限制、单行线、道路等级等业务规则如何在图模型中表达？低空/水下路网有什么不同？

💡 带着这些问题，我们开始今天的内容

核心算法清单

本次课核心算法——路网模型与拓扑构建

★★★	**图的存储结构**	邻接表/前向星/边表——路网基础数据结构，O(1)查询邻接边。前向星最省内存，邻接表最灵活，边表最简单
★★★	**节点合并算法**	基于空间距离阈值的节点去重——解决OSM同一交叉口多节点问题。O(n log n)用空间索引加速。snap容差2-5米
★★★	**边拆分算法**	在交叉口处拆分道路——将长Way拆分为独立边。O(n)遍历，按节点度数判断拆分点。拓扑构建的前提
★★☆	**拓扑构建算法**	从节点和边生成邻接表。O(n+m)时间复杂度。构建入边/出边邻接表，支持有向图
★★☆	**转向限制建模**	禁止左转/右转的图表达。通过虚节点或边属性实现。O(1)查询。OSM restriction relation解析
★★☆	**道路权重计算**	长度/速度/通行时间。Haversine公式计算球面距离，O(1)每条边。按道路等级推断默认限速

低空与水下路网建模——三维空间与时变网络

低空无人机航路

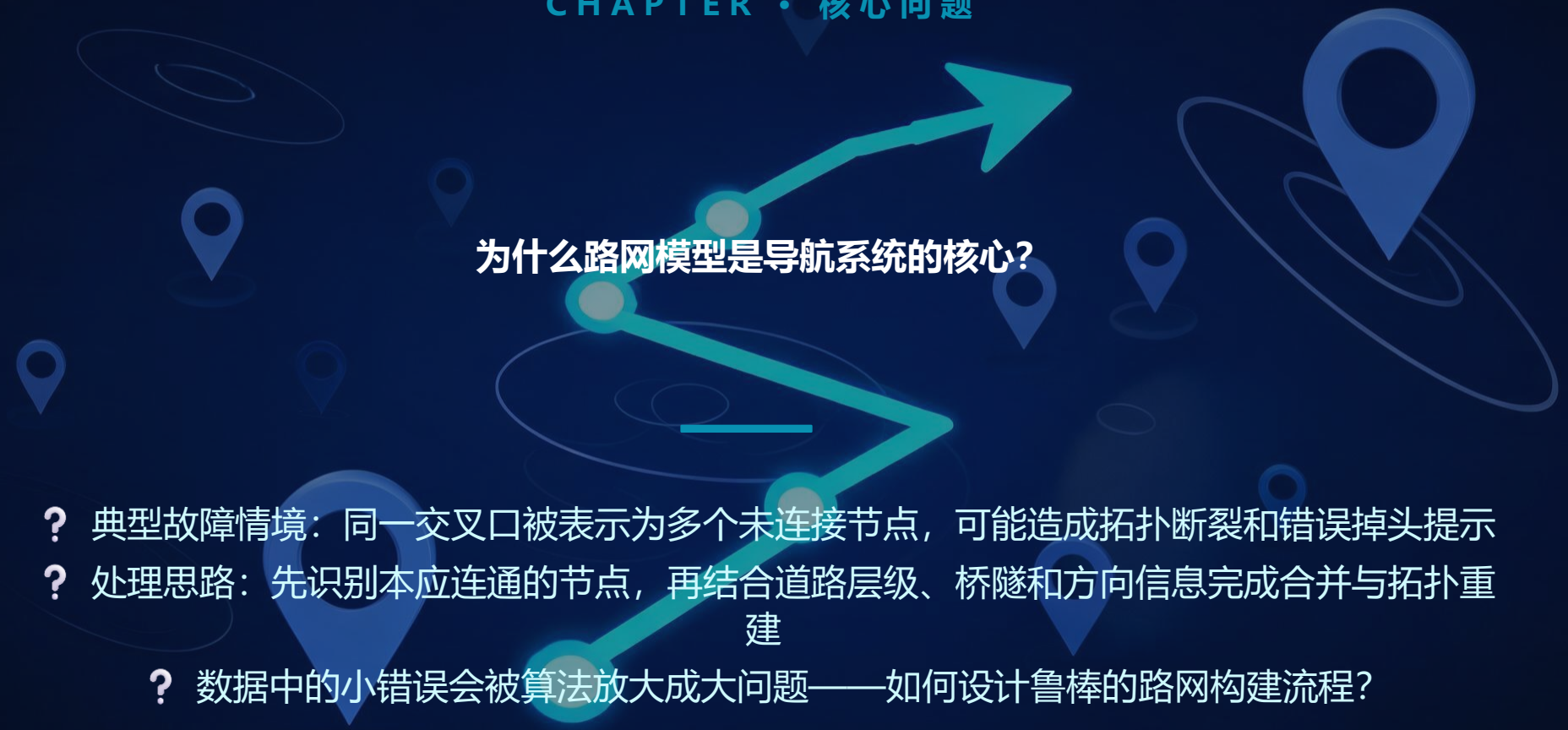
- 使用三维坐标表示航路点，并离散为多个高度层
- 边权同时考虑巡航、爬升和下降成本
- 禁飞区、限飞区和障碍物作为空间约束
- 航路点连接成航线网络
- 高度层可按航向分配，减少飞行冲突
- 路径规划可采用RRT、RRT*或人工势场法
- 这类问题属于运动规划，不是普通图搜索

VS

水下AUV航路

- 洋流使路网边权随时间变化
- 顺流降低通行时间，逆流增加通行时间
- 时变路网需要专门的最短路径算法
- 水下空间按深度分层，不同层的洋流和声速不同
- 地形匹配导航用水深序列匹配海底DEM
- 其思路与地面HMM地图匹配相似

CHAPTER · 核心问题



为什么路网模型是导航系统的核心？

- 典型故障情境：同一交叉口被表示为多个未连接节点，可能造成拓扑断裂和错误掉头提示
- 处理思路：先识别本应连通的节点，再结合道路层级、桥隧和方向信息完成合并与拓扑重建
- 数据中的小错误会被算法放大成大问题——如何设计鲁棒的路网构建流程？

什么是路网模型？——道路网络的数学抽象

1 定义

- 路网模型用节点和边表达道路拓扑
- 节点表示可转向位置，边表示可通行路段

2 权重

- 边权可表示距离、时间、费用或能耗
- 路径规划依据权重选择路线，多目标任务可使用权重向量

3 路网模型 ≠ 地图数据

- 地图数据描述坐标、形状和显示效果
- 路网模型描述连通关系、方向、权重和规则

路网建模的核心决策——有向图还是无向图?

无向图

- 双向边模型简单，适合规则较少的局部路网
- 单行线和转向限制难以直接表达
- 生产导航通常将双向道路拆成两条有向边

vs

有向图

- 每条边都有明确方向
- 单行线和转向限制可以直接建模
- 适合城市道路和复杂交叉口
- 生产系统通常统一使用有向图
- 双向道路用方向相反的两条边表示
- 低空和水下路网也通常采用有向图

图论基础回顾——大三学生需要掌握的图论概念

基本概念

- 图G=(V,E)，V是顶点集合，E是边集合
- 有向图的边是有序对，无向图的边是无序对
- 加权图为每条边设置权重
- 顶点度数表示关联边数
- 有向图区分入度和出度
- 路径由相邻顶点序列组成
- 最短路径的权重和最小
- 连通图中任意两点之间都有路径

路网相关概念

- 交叉口通常是度数不小于3的节点
- 道路端点通常是度数为1的节点
- 连通分量表示内部互相可达的子图
- 有向图还要分析强连通分量
- 邻接矩阵占用$O(V^2)$空间
- 邻接表占用$O(V+E)$空间
- 路网通常较稀疏，适合使用邻接表

从OSM到路网：6步基础构建，再补充转向限制建模



Step 1

数据读取

读取G1组输出的nodes.csv和ways.csv，解析标签和几何

→



Step 2

节点筛选

筛选道路节点，过滤非道路Node，提取Way的节点引用

→



Step 3

节点合并

依据数据精度设置候选容差，并结合道路层级、桥隧与方向约束避免误合并

↓



Step 6

拓扑构建

构建邻接表(入边/出边)，建立节点-边的双向索引

←



Step 5

权重计算

Haversine算距离，按道路等级推断限速，计算通行时间权重

←



Step 4

边拆分

在交叉口处拆分长Way为独立边，按节点度数判断拆分点

6+1

基础流程+限制建模

分步分析

时间复杂度

需调参

snap容差

实测

数据规模

节点合并算法——如何识别"同一个交叉口"?

1 问题本质

- 同一交叉口可能因不同Way录入而出现多个近邻节点
- 节点不共享会造成拓扑断裂和路径不连通

2 算法原理

- 距离小于阈值的候选节点可判为同一交叉口
- 保留一个节点并重定向其他节点的引用
- 合并标签属性，并记录映射关系

3 性能优化

- 网格索引或R树先筛选候选节点对
- 再计算精确距离，避免$O(n^2)$暴力比较
- 也可按纬度排序后使用滑动窗口

节点合并进阶——自适应阈值与连通性约束

自适应阈值	不同道路等级用不同阈值：主干道交叉口大(5米)，支路小(2米)。交叉口节点度数高的阈值大，端点阈值小
基于连通性的合并	合并前检查：合并后是否会导致错误的连通？如平行道路的节点距离近但不应合并。可增加约束：只有属于不同Way且在道路端点的才合并
迭代合并	一次合并后可能产生新的近距离节点对，需要迭代直到没有可合并对。通常2-3轮即可收敛
合并记录与回滚	记录每次合并：被合并节点id→保留节点id，方便问题溯源和回滚。合并后要更新所有引用该节点的Way和Relation
交叉口检测	degree>=3的节点是交叉口。交叉口节点合并要更谨慎(影响多条路)，端点合并可更激进

常见错误：snap太大误合并平行道路/立交桥上下层合并/交叉口节点过度合并导致信息丢失

边拆分算法——如何将长道路切成独立边?

1 为什么要拆分

- OSM Way可能跨越多个交叉口
- 路径规划需要把相邻交叉口之间的路段拆成独立边

2 拆分点判断

- 交叉口和道路端点必须作为拆分点
- 限速或道路等级变化处也可拆分
- 遍历Way节点并依据全局度数判断拆分位置

3 拆分过程

- 每段子边继承原Way的标签属性
- 为子边分配唯一ID并重新计算长度和权重
- 一遍扫描即可完成基础拆分

道路权重计算——距离、速度、通行时间怎么算?



距离权重
最基础的权重

计算	Haversine计算球面距离
精度	不反映实际通行速度
限速推断	无需限速数据
默认值	直接计算



时间权重
最常用的权重

计算	距离÷速度
精度	更符合驾驶时间
限速推断	优先读取maxspeed；缺失时按规则补全
默认值	示例值需结合地区法规与样本校准



多目标权重
高级路径规划

计算	按距离、时间和费用加权
精度	可按需求调整偏好
限速推断	需要多类属性
默认值	按应用场景设定

小范围计算可先投影到适合的平面坐标系；误差取决于范围、纬度和投影，需用样本验证

拓扑关系的三种表达——邻接表、前向星、边表

① 边表 Edge List
最简单直观

结构	边数组：(from, to, w)
空间	O(m)，简单紧凑
查询邻接边	O(m)遍历所有边，较慢
增删边	O(1)追加，速度快
适用	数据存储和交换

② 邻接表 Adjacency List
最灵活常用

结构	每个顶点维护邻接边
空间	O(n+m)，有指针开销
查询邻接边	O(degree(v))，直接访问
增删边	取决于底层容器
适用	动态图和路径规划

③ 前向星 Forward Star
最省内存

结构	边数组和顶点起始索引
空间	O(n+m)，无指针开销
查询邻接边	按索引范围访问
增删边	O(m)重建，更新成本高
适用	静态大路网/内存受限

课程项目优先用邻接表，便于实现与调试；大规模静态路网可选前向星或CSR，以换取更紧凑的内存布局。

拓扑构建算法——从节点和边生成邻接表

01	**初始化**	创建空的入边邻接表和出边邻接表，大小为节点数。in_adj = [[] for _ in range(n)], out_adj = [[] for _ in range(n)]
02	**遍历边表**	逐条处理每条边。对有向边<u,v>，将边加入out_adj[u]（u的出边）和in_adj[v]（v的入边）
03	**无向边处理**	如果是无向边(u,v)，拆成两条有向边：u→v和v→u。分别加入对应的出边和入边邻接表
04	**排序(可选)**	每个顶点的邻接边按目标节点id排序，方便二分查找特定边。或按权重排序，方便贪心选择
05	**构建反向索引**	边id→(from,to,weight,attrs)映射表。通过边id快速查边的详细信息。路径规划输出边序列后查表得道路信息
06	**度数统计**	统计每个节点的入度、出度、总度数。degree[i] = len(out_adj[i])。用于识别交叉口(degree>=3)和端点(degree=1)
07	**验证**	连通性检查：BFS/DFS看最大连通分量占比。边数校验：出边总数应等于边数(有向图)或2倍边数(无向图)

转向限制建模——禁止左转/右转如何在图中表达?

1

方法1：边属性法

- 为每条入边维护禁止转向的出边集合
- 存储直观，但路径搜索每一步都要检查限制
- 适合直接表达OSM转向限制

2

方法2：虚节点法

- 把交叉口拆成多个虚节点
- 仅在允许转向的虚节点之间连边
- 普通Dijkstra或A*可直接使用，但图规模会增大

3

OSM restriction relation

- restriction关系由from、via和to成员组成
- 解析时校验成员角色、几何连接和行驶方向
- 课程项目需要输出可追溯的转向限制表

单行线与双向道路——方向属性的处理

单行线 (oneway=yes)

- oneway=yes时按Way节点顺序通行
- oneway=-1时按Way节点逆序通行
- oneway=no时按双向道路处理
- 环岛和部分高速道路通常按单向处理
- 单向道路只生成一条有向边
- 建边前先统一方向规则

双向道路 (oneway=no)

- 双向道路生成方向相反的两条边
- 两条边可共享道路属性
- 不同方向可以设置不同权重
- 缺少oneway标签时按项目规则判断
- 统一使用有向图可减少特殊分支
- 往返边可用关联ID配对

道路等级与功能分类——路网的层次结构

motorway(高速)	封闭、出入口受控的道路类别。通行规则和限速应读取maxspeed=*、access=*等标签，并结合地区法规
trunk(城市快速路/国道主干)	高等级主干道路。highway=*描述道路功能，不等同于统一的法定限速
primary(主干道)	重要干线道路。实际通行速度与限速需从数据标签或外部权威数据获得
secondary(次干道)	次级连接道路。建模时保留道路等级，并单独维护速度和访问规则
tertiary(支路)	连接局部区域的道路类别。具体含义按当地OSM标注规范解释
residential(居民区道路)	以居住区到达为主的道路类别。不能仅凭类别推断精确速度
service/unclassified	服务道路(停车场/园区内部)和未分类道路。最低等级，路径规划中优先级低

连通分量分析——如何识别路网中的"孤岛"？

1 为什么要分析

- 最大连通分量占比可作为质量指标
- 没有统一阈值，应结合区域和道路类型解释异常分量

2 算法

- BFS或DFS可标记每个连通分量
- 遍历复杂度为O(n+m)
- 有向图需要区分强连通分量和弱连通分量

3 结果应用

- 与课程基线或上一版本比较
- 结合道路类型说明异常分量的形成原因

OSRM路网构建流程——工业级开源导航引擎

现象 OSRM是开源路由引擎。其处理流程包含数据提取、路网构建、预处理和查询服务，适合作为工程案例。

原因分析

数据提取：读取OSM PBF，筛选可通行道路。

路网构建：处理节点、边和道路属性，生成有向图。

拓扑存储：组织出入边，保留单行线和转向限制。

预处理：构建查询加速结构，性能按数据规模实测。

模式配置：为不同交通模式设置通行规则和速度。

工程启示

工程启示：把业务规则放入可配置脚本；预处理负责构建查询加速结构；同一框架可服务驾车、步行和骑行等模式。存储结构需结合规模和更新频率选择。

工程案例

商业导航路网的常见工程设计

现象

商业导航路网通常增加更细的道路属性、交通规则和更新机制。具体规模与完整率以公开资料或实测数据为准。

原因分析

道路分级：用于分层搜索，级数由数据模型定义。

丰富属性：按规范维护限速、车道、收费和桥隧等信息。

转向建模：记录转向规则，可扩展车道和信号控制。

增量更新：基础路网与交通状态分开维护版本。

多种模式：不同交通方式使用独立规则与权重。

工程启示

商业数据与OSM在许可、覆盖、属性和更新机制上不同；比较时应统一区域、时间和指标。

低空无人机导航：三维航路网络建模

1 地面 vs 低空

- 地面路网是二维拓扑，低空路网增加高度和空域约束
- 低空数据包括地形、建筑、空域和障碍物
- 固定道路边界变为可通行空间与管制规则

2 三维航路网络建模方法

- 高度分层法把三维空间离散为多层二维路网
- 层间用爬升和下降边连接，便于复用二维算法
- 三维路标图在自由空间采样并进行碰撞检测
- 分层法简单，路标图精度更高但计算量更大

3 空域约束

- 禁飞区禁止进入
- 限飞区设置审批或高度条件
- 空中走廊和高度层规则限定可选航路

水下AUV导航：洋流影响的时变网络建模

1 水下导航的特殊性

- 水下导航依赖INS和声学定位
- 洋流改变航行速度和能耗
- 深度与声速分层影响通信和定位

2 时变图(Time-Dependent Graph)

- 边权是随出发时间变化的函数
- 顺流与逆流会改变同一路段的通行时间
- 固定权重的经典Dijkstra不能直接处理时变网络

3 时变最短路径算法

- 离散时间片方法简单，但精度受时间粒度限制
- 连续时间模型可在FIFO假设下改进Dijkstra
- 预测模型可支持AUV实时调整速度和航向

路网数据存储格式设计——road_nodes.csv和road_edges.csv

road_nodes.csv 节点表

- id：节点唯一编号，建议连续编码
- lat、lon：WGS84经纬度坐标
- degree：节点连接边数
- osm_id：原始OSM节点编号，便于溯源
- tags：节点标签属性
- 交叉口、端点和中间点可依据degree区分

road_edges.csv 边表

- id、from_node、to_node：边及端点编号
- length：边长，可由形状点累加
- maxspeed：限速或推断速度
- travel_time：按长度和速度计算
- highway、name：道路等级与名称
- oneway：单行方向标记
- osm_way_id：原始Way编号
- lanes、surface：可选扩展属性

路网数据交换标准——NDS、OpenDRIVE、GeoJSON对比



NDS
导航数据标准

用途	车载导航数据格式
特点	分层瓦片、增量更新、权限保护
精度	道路级，含属性与拓扑
开放性	成员许可，具体授权以规范为准
适用	量产车载导航



OpenDRIVE
自动驾驶标准

用途	自动驾驶仿真与测试路网
特点	车道级，含标线、标志和信号灯
精度	车道级，精度依数据而定
开放性	ASAM标准，具有公开规范和工具
适用	自动驾驶开发与仿真



GeoJSON
通用GIS标准

用途	通用GIS交换格式，适合Web
特点	人可读、跨平台、生态丰富
精度	精度取决于数据源
开放性	开放标准，可自由实现
适用	Web地图、数据交换与课程项目

课程项目可用CSV保存节点与边，用GeoJSON完成可视化和交换；车载导航与自动驾驶项目分别参考NDS和OpenDRIVE等标准。

路网更新与版本管理——道路变化了怎么办?

全量更新	定期(每周/每月)重新构建完整路网。简单可靠，但更新慢、用户需下载全量数据。适合大版本更新
增量更新	只更新变化的部分(新增/删除/修改的路段)。用户端只需下载增量包(全量的5-10%)。需要变化检测、依赖处理、版本管理。适合小版本更新
变化检测	对比两个版本OSM数据，检测新增/删除/修改的元素。基于元素ID匹配+属性差异+几何差异。O(n log n)时间复杂度
依赖关系	修改一条道路可能影响：关联的转向限制Relation、关联的POI、关联的行政边界、交通信号灯、公交路线。增量更新要处理依赖，保证一致性
版本号	语义化版本：主.次.修(如1.2.3)。主版本：数据模型变更(不兼容)。次版本：新增道路(向下兼容)。修订版本：修复错误
实时更新	交通事件(事故/拥堵/施工封路)是实时变化的。不是修改路网拓扑，而是在边权上叠加实时路况权重。属于G6导航引擎的实时交通模块

性能优化技巧——如何让路网构建更快?



O(nlogn)

时间复杂度

节点合并且空间索引(R树/网格索引)，不要O(n²)暴力。边拆分O(n)一遍扫描。拓扑构建O(n+m)



CSR

存储格式

通常比对象化邻接结构更紧凑且更利于顺序访问，具体节省比例需用同一数据和实现做基准测试



整型ID

编号优化

连续编号便于数组索引，通常能减少散列表和对象开销；实际收益取决于语言与实现



实测

内存对比

坐标存储需先确定精度要求。可采用整数定点量化或局部投影坐标，再通过误差测试选择数据类型

常见bug与调试技巧——路网构建中最容易犯的错误

现象

路网构建中的错误通常不会报错，却会直接影响路径结果，因此需要系统化验证。

原因分析

编号不连续：重编号并保存映射，避免邻接数组越界。

方向错误：处理oneway，并用可视化检查逆行边。

限制遗漏：按成员角色验证from、via、to的连通。

snap过大：减小阈值，并增加道路与层级约束。

长Way未拆分：检查交叉口和属性变化点。

工程启示

调试流程：先可视化检查断开、错连和方向，再统计节点、边和连通分量；最后抽样验证路径与属性，并与可靠基线比较。

路网质量评估体系——如何判断构建的路网好不好?

1 拓扑质量

最大连通分量占比、孤立路段数量、异常端点和转向限制有效率。阈值由数据范围与课程验收标准确定

2 属性完整性

道路名称、限速、车道数和路面类型的完整率。先报告缺失率，再说明补全规则和数据来源

3 几何准确性

道路长度、节点位置和形状保真度应与参考数据或抽样真值比较；验收阈值需在项目开始前明确

故障案例

路网构建的典型故障情境与排查方法

现象　以下情境用于课堂排查练习，不代表某个真实平台的公开事故记录

原因分析

节点未合并：检查端点距离和拓扑连通，避免绕行。

单行方向错误：核对oneway与边方向，防止逆行。

转向限制遗漏：核对from、via、to的连通关系。

snap过大：减小阈值，并增加道路属性约束。

长Way未拆分：检查交叉口和属性变化点。

工程启示

关键属性要进行规则校验、抽样核对和回归测试，并记录仍未覆盖的情况

课堂讨论

路网建模中的开放性问题——有标准答案吗？

观点1 snap阈值到底设多少？——设太小合并不了，设太大误合并。不同城市/不同道路等级应该不同吗？自适应阈值如何设计？

观点2 路网应该有多详细？——小区内部道路、停车场内部通道要不要加进去？越详细越准确但数据量越大、计算越慢。粒度如何平衡？

观点3 转向限制建模用虚节点还是边属性？——各有优劣。什么场景用虚节点？什么场景用边属性？有没有更好的方法？

观点4 路网数据如何快速更新？——道路每天都在变化(新建/施工/封路)。全量更新太慢，增量更新处理依赖复杂。如何高效地保持路网新鲜度？

💡 **讨论引导**

讨论要点：这些都是实际工程中没有标准答案的问题，需要根据具体场景(数据规模/性能要求/精度要求)做权衡。工程师的核心能力就是在约束条件下做最优决策

G2组课程项目详细指导——路网模型组具体做什么?

01	数据读取	读取G1组输出的nodes.csv和ways.csv。解析标签，筛选道路数据(highway标签的Way)
02	节点合并	节点合并：根据数据精度设置候选容差，并用网格或R树减少比较；参数需通过样本验证。
03	边拆分	在交叉口(degree>=3)处拆分长Way为独立边。保留道路属性(highway/name/maxspeed/lanes/oneway)
04	权重计算	Haversine公式计算每条边的长度。按道路等级推断默认限速。计算通行时间权重
05	拓扑构建	构建邻接表(出边+入边)。处理oneway方向。构建节点-边双向索引。节点重编号(连续整型)
06	转向限制	解析OSM restriction relation(from/via/to)。构建转向限制表。路径规划时检查限制
07	质量检查	连通分量分析(最大分量占比)、属性完整率、边数统计、可视化验证。生成质量报告

接口定义与数据交付标准——G2组与上下游组的数据契约

G1→G2 输入

- nodes.csv：节点编号、坐标和标签
- ways.csv：Way编号、节点引用和标签
- tags.csv：可选的标准化标签表
- 坐标系统一为WGS84
- 文件编码统一为UTF-8
- 输入数据应完成标签标准化与坐标修正

G2→G3/G4 输出

- 输出road_nodes.csv、road_edges.csv和restrictions.csv
- 字段名、字段类型和编码遵循接口文档
- 报告连通性、异常端点和属性完整率
- 按照教师给定的验收标准检查

FAQ — 学生最常问的问题

Q: 路网构建后连通率很低怎么办?

A: 先检查节点合并是否正确(snap是否太小)，再检查边拆分是否正确(degree>=3的节点都拆分了吗)，最后检查oneway处理是否正确。可视化路网定位问题区域

Q: 节点合并总是误合并平行道路?

A: 降低snap阈值(如从5米降到2米)，增加约束条件：只有道路端点才合并/只有不同Way的节点才合并/合并前检查道路名称是否一致

Q: Python处理500万节点太慢?

A: 1) 先过滤：只保留道路相关的节点(被highway Way引用的)，通常只有几十万；2) 用numpy/pandas向量化操作；3) 用空间索引(Rtree库)加速邻近查找；4) 核心循环用C扩展或Numba加速

Q: 转向限制relation解析失败?

检查from、via、to成员是否按行驶方向连续连接；via既可能是节点，也可能由一条或多条way组成，不能只检查Way的首尾节点

Q: 如何验证路网构建是否正确?

A: 1) 统计指标对比：节点数/边数/连通率与预期对比；2) 可视化：用QGIS加载路网，肉眼检查；3) 路径规划测试：选几条已知路线看规划结果是否合理；4) 对比参考：与OSRM的结果对比

Q: 路网数据需要多大内存?

内存占用取决于节点、边、属性字段和Python对象开销。应在目标数据上测量；需要压缩时可用连续ID、数组或CSR结构。

伦理责任

路网构建的伦理与社会责任——技术背后的人文关怀

1 数据偏见与公平性

- 数据覆盖差异会影响不同区域的导航质量
- 建模与评估应关注欠覆盖区域，避免扩大数字鸿沟

2 导航与城市交通

- 最快路线可能把交通引入居民区
- 道路等级和访问限制需要兼顾个体效率与公共影响

3 应急救援与路网

- 应急车辆需要特殊通行规则和信号优先信息
- 路网数据应支持应急通道的可靠验证

低空/水下路网建模总结对比——多模态导航的路网差异



地面(2D)
道路网络

空间维度	二维(x,y) + 拓扑
边的含义	固定道路与静态规则
权重	距离、时间或费用
障碍物	道路边界和禁行区
建模方法	成熟的有向加权图



低空(3D)
航路网络

空间维度	三维(x,y,z) + 空域
边的含义	航线、空中走廊与管制区
权重	距离、时间、能耗和高度
障碍物	建筑、塔、电线和禁飞区
建模方法	分层航路图或PRM



水下(3D)
时变网络

空间维度	三维(x,y,z) + 深度
边的含义	洋流、声速与时变环境
权重	时间、能耗和洋流影响
障碍物	海底、沉船、管道和温跃层
建模方法	时变图与流场建模

三类路网都用节点、边和权重表达。主要差异来自空间维度、约束是否随时间变化，以及障碍物类型。

拓扑构建进阶——前向星实现与压缩存储

前向星 Forward Star

- edge[]保存边，head[]保存每个节点的首条边索引
- 边记录目标节点、权重和下一条边索引
- 按next索引遍历同一起点的出边
- 连续数组存储紧凑且缓存友好
- 适合大规模静态路网
- 动态增删边较困难，构建时需要排序

CSR 压缩稀疏行

- offsets[]记录每个节点的边起始位置
- edges[]与weights[]连续保存目标节点和边权
- 相邻offset区间就是一个节点的出边
- 空间占用紧凑，邻接遍历效率高
- 构建复杂，更新成本高
- 适合完全静态的大规模路网
- 工业路由引擎常使用CSR或类似结构

路网可视化技术——如何画出漂亮的路网图



QGIS

最强大的桌面GIS。加载GeoJSON/Shapefile，按道路等级设置不同颜色和线宽，制作专业级路网图



Leaflet

Web地图库，轻量级。用GeoJSON叠加路网，支持交互(点击查看属性)，适合网页展示



Matplotlib/NetworkX

Python绘图。NetworkX自带draw()函数，适合小路网快速可视化和算法调试



kepler.gl

Uber开源的地理空间可视化工具。支持大数据量可视化，酷炫的3D效果，适合展示演讲

大规模路网处理技巧——全国级数据如何处理?

1 分而治之

按区域(省/市)分片处理，最后合并。每片数据量小，处理快，可并行。合并时处理边界处的道路连接。Mapbox/OSRM都用分片策略

2 流式处理

不把全部数据加载到内存，逐条读取/处理/输出。osmium就是流式处理。Python可用生成器(yield)实现流式。内存占用与数据量无关，只与当前状态有关

3 并行处理

节点合并/边拆分等无依赖操作可多线程/多进程并行。用Python的multiprocessing或concurrent.futures。注意GIL限制，CPU密集型用多进程而非多线程

路网数据压缩——如何让路网文件更小?

坐标量化	可将经纬度按既定精度量化为整数；OSM PBF通过granularity/offset恢复坐标，并在DenseNodes中使用差分编码
差分编码	节点ID/坐标按顺序排列，存储与前值的差(差值小，用更少的字节)。OSM PBF的DenseNodes就是用差分+Varint编码
字符串表	道路名称/标签等字符串去重，用整数索引引用。高频字符串(highway/primary/residential等)只存一次。OSM PBF的StringTable
折线简化	道路形状点用Douglas-Peucker算法简化，减少顶点数。不同缩放级别用不同精度的简化结果
通用压缩	gzip/zstd等通用压缩算法。路网数据重复性高，压缩率可达50-70%。传输和离线包常用

课程项目规模的数据(武汉10万边)不用压缩也完全没问题，但了解这些技术对理解工业级实现有帮助

多源路网数据融合——OSM+商业地图+轨迹数据

融合目的与挑战

- 融合目标是补充单一来源的覆盖与属性
- OSM开放且覆盖广，但部分属性可能缺失
- 商业数据属性较全，但许可和使用受限
- 融合前必须统一坐标系
- 同一道路需要结合空间、属性和拓扑匹配
- 不同来源的更新频率也要统一管理

融合方法

- 实体匹配后选择更完整、可信的属性
- 轨迹对齐可校正道路位置并发现新增道路
- 主辅数据源方案便于控制融合范围
- 匹配阈值需要通过样本验证
- 记录每个字段的来源和更新时间
- 使用数据时必须遵守各自许可证

路网变化检测与增量更新——如何只更新变化的部分?

1 变化检测方法

- 相同ID用于检查标签、几何和版本变化
- 新ID表示新增元素，旧ID消失表示删除元素
- version和timestamp可辅助判断变化

2 增量更新流程

- 检测新增、删除和修改的道路与节点
- 处理受影响的转向限制和关联关系
- 生成带操作类型的增量包
- 客户端应用增量并更新本地路网
- 更新后重新验证连通性和属性

3 工程挑战

- 维护关联数据之间的依赖关系
- 控制增量包大小和版本兼容性
- 保留失败后的回滚能力

路网构建的测试方法——如何保证代码和数据的质量?

单元测试	每个函数单独测试：节点合并函数(给定节点列表和阈值，验证合并结果)、边拆分函数(给定Way和节点度数，验证拆分结果)、权重计算函数(给定坐标，验证距离计算正确)
集成测试	整个构建流程端到端测试：输入测试数据，输出路网，验证节点数/边数/连通率/抽样边属性。用已知正确结果的小数据集做回归测试
属性测试	随机抽样验证：随机选100条边，手动检查属性(name/highway/maxspeed)是否正确。检查单行线方向是否与实际一致。检查转向限制是否正确解析
拓扑测试	连通性测试：已知连通的两点，检查路网中是否可达。可达性测试：随机选100对节点，检查路径规划是否能找到路径(合理的路网应该>99%可达)
性能测试	不同规模数据的构建时间/内存占用。大数据量下是否会OOM。时间复杂度是否符合预期(O(n log n))
回归测试	每次代码修改后跑全套测试，确保没有引入新bug。保存已知正确的输出作为基准，每次对比

参考文献与延伸阅读

书籍与教程

- 《算法导论》图论章节
- 《Network Science》
- 《GIS Algorithms》
- 《Spatial Networks》
- LeetCode图论专题

开源项目

- OSRM：C++开源路由引擎
- GraphHopper：Java路由引擎
- NetworkX：Python图论算法库
- igraph：高性能图论库
- osmnx：OSM路网下载与构建
- pandana：城市网络分析库

论文与标准

- Haklay 2010：OSM质量研究
- Geisberger 2008：收缩层次算法
- NDS导航数据标准
- ASAM OpenDRIVE标准
- OSM Wiki Map Features

本章小结

本次课核心判断

🎯 核心判断

可拆为6步基础构建，再单独处理转向限制建模

三大核心算法

- 节点合并控制交叉口连通
- 边拆分生成可转向路段
- 拓扑构建形成邻接结构

业务规则建模

- restriction关系表达转向限制
- 有向边表达单行规则
- 道路等级支持分层搜索

质量与性能

- 检查连通性和属性完整率
- 用空间索引与连续ID提升性能
- 验收阈值以课程标准为准

多模态拓展

- 低空路网增加高度和空域约束
- 水下路网增加洋流和时间变化
- 两类扩展都建立在图论抽象上

本章小结

本次课核心判断

核心判断

G2组把G1的点线数据转换为可用于路径规划的路网拓扑，输出质量直接影响后续模块。

工程思维

- snap阈值影响错连与断连
- 方向和转向规则影响路径合法性
- 存储结构影响查询性能

质量意识

- 构建后检查连通性
- 统计属性完整率
- 通过可视化和抽样路径验证

工具能力

- 掌握NetworkX等图论工具
- 使用空间索引和距离计算
- 用QGIS或Leaflet检查路网

协作意识

- 输入格式与G1组保持一致
- 输出格式满足G3和G4组需求
- 开发前先确认接口定义

参考文献与延伸阅读

图论与算法

- Dijkstra 1959：最短路径
- Cormen等：《Introduction to Algorithms》
- Sedgewick：《Algorithms》
- Hart等 1968：A*算法
- Geisberger等 2008：Contraction Hierarchies

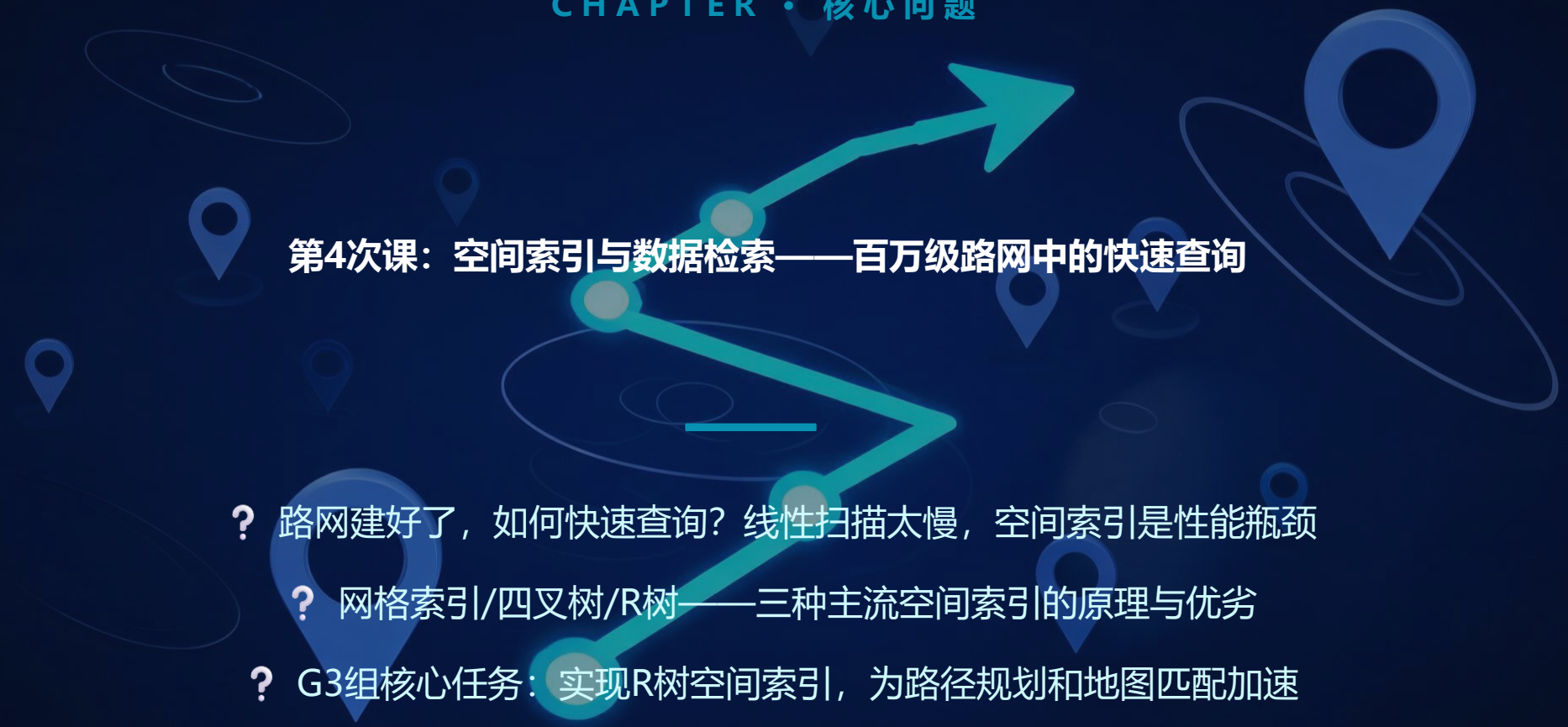
路网建模

- OSM Wiki Map Features：道路标签
- Goodchild：《Geographic Information Systems and Science》
- Leduc 2008：道路交通数据
- OSRM官方文档
- GraphHopper官方文档

多模态导航

- Karaman与Frazzoli 2011：RRT*
- Khatib 1986：人工势场
- Meduna 2007：地形匹配导航
- OpenDroneMap：无人机三维重建

CHAPTER · 核心问题



第4次课：空间索引与数据检索——百万级路网中的快速查询

? 路网建好了，如何快速查询？线性扫描太慢，空间索引是性能瓶颈

? 网格索引/四叉树/R树——三种主流空间索引的原理与优劣

? G3组核心任务：实现R树空间索引，为路径规划和地图匹配加速

路网构建工具速查表——常用工具和库一览

NetworkX — Python图论算法库，功能最全，适合学习和小规模数据。draw()可视化/最短路径/连通分量

igraph — 高性能图论库(C核心，Python/R绑定)，比NetworkX快10-100倍，适合大规模数据

osmnx — Python，从OSM下载+构建路网+可视化一体化。G2组可以参考其构建逻辑

Rtree — Python空间索引库(R树实现)，用于节点合并加速。G3组也会用到

Shapely — Python几何操作库，计算距离/相交/包含等，用于几何验证和分析

QGIS — 桌面GIS软件，路网可视化/空间分析/质量检查的必备工具

OSRM — 工业级开源导航引擎(C++)，路网构建和路径规划的标杆实现，值得深入研究

CHAPTER · 核心问题

从理论到实践——路网构建的核心思想

6步基础流程 + 转向限制 + 3种存储

需要6步基础构建，并补充转向限制建模

第1次作业总览——路网拓扑构建基础（G2组第1次作业）

任务1

□ 约2小时

- 读取并筛选G1组数据
- 统计节点、道路等级和里程

任务2

□ 约4小时

- 实现基于距离的节点合并
- 使用网格索引或R树加速

任务3

□ 约3小时

- 识别交叉口并拆分长Way
- 计算边长、速度和通行时间

任务4

□ 约3小时

- 构建出边与入边邻接表
- 解析并输出转向限制

任务5

□ 约2小时

- 统计连通性和属性完整率
- 抽样路径并提交质量报告

提交 第4次课前提交：road_nodes.csv、road_edges.csv、restrictions.csv、源码、质量报告和README。数据包供G3/G4组使用，扩展任务另附说明。

第1次作业任务1详解——数据读取与节点筛选

步骤1
□ 读取
- 读取nodes.csv和ways.csv
- 统一使用UTF-8编码

步骤2
□ 筛选
- 保留带highway标签的Way
- 统计各道路等级数量和里程

步骤3
□ 提取
- 提取道路Way引用的节点ID
- 过滤未被道路引用的节点

步骤4
□ 统计
- 统计节点总数、引用数和度数
- 输出道路等级分布

步骤5
□ 探索
- 用NetworkX检查节点数和边数
- 用QGIS发现明显的数据问题

提交 提交：探索性分析报告，包含统计表格、度数分布图、可视化截图。为后续节点合并和边拆分做准备

第1次作业任务2详解——节点合并（核心难点）

方案1

□ 简单版

- 计算所有节点对距离
- 仅用于小数据或正确性基线

方案2

□ ★推荐

- 按阈值划分空间网格
- 只比较相邻网格内的候选节点

方案3

□ 进阶版

- R树先查询邻近候选
- 再用精确距离判断是否合并

合并策略

□ 通用

- 保留代表节点并重定向引用
- 记录old_id到new_id映射

验证

□ 必备

- 比较合并前后的节点数量
- 可视化检查误合并并验证连通性

提交 评分重点：算法时间复杂度(用空间索引加速)、合并策略的合理性、验证方法的完整性。加分项：自适应阈值/迭代合并/连通性约束

课程作业

第1次作业任务3详解——边拆分与属性计算

拆分点

□ 判断

- 交叉口和道路端点作为拆分点
- 属性变化点按项目需要拆分

拆分过程

□ 遍历

- 遍历Way并在拆分点结束当前边
- 每段子边继承原Way标签

边编号

□ 策略

- 为每条边分配连续唯一ID
- 保留osm_way_id和往返边关联

长度计算

□ Haversine

- 累加形状点之间的距离
- 保留中间点形成的完整几何

权重计算

□ 推断

- 优先读取maxspeed，缺失时按规则补全
- 计算通行时间并标记推断字段

提交 提交拆分后的边表，保留from、to、length、maxspeed、travel_time、highway、name和oneway等字段；抽样检查边数与长度。

第1次作业任务4详解——拓扑构建与转向限制

节点重编号

□ 0..n-1

- 将节点ID重编号为连续整数
- 保存旧ID到新ID映射

邻接表构建

□ O(n+m)

- 遍历边构建out_adj和in_adj
- 双向道路加入两个方向

转向限制解析

□ restriction

- 读取restriction的from、via和to
- 校验成员角色、方向与连接关系

输出格式

□ 标准化

- 输出节点、边和转向限制文件
- 字段与编码遵循接口文档

验证

□ 必做

- 核对边数与邻接表记录数
- 分析连通分量并抽样测试路径

提交 提交路网数据、邻接表和转向限制表；文件格式严格遵循接口定义，供G3/G4组直接使用。

评分标准

第1次作业考核打分——明确评分维度和优秀标准



功能完整性(30%)

要求的功能是否全部实现，能否正常运行。优秀：全部功能实现+运行稳定。及格：基本功能可用



代码质量(20%)

代码结构、命名规范、注释、模块化。优秀：代码优雅+模块化好。及格：代码可读



可视化+文档(25%)

可视化效果(15%)+文档质量(10%)。优秀：界面友好+文档详尽。及格：有基本可视化+文档



AI日志+答辩(25%)

检查拓扑结构和数据质量。优秀作品应包含邻接结构、转向限制、连通性分析、可视化验证和完整质量报告。

常见问题

FAQ — 学生最常问的问题

Q: G1组的数据还没出来怎么办?

- 先用OSM原始数据或课程示例数据开发
- 接口对齐后再替换为G1组正式数据

Q: 节点合并用什么方法?必须用R树吗?

- 网格索引简单有效，可作为基础实现
- R树属于进阶方案，重点是理解候选筛选思路

Q: 转向限制必须实现吗?

- 基础要求是解析并保存restriction
- 虚节点建模可作为扩展任务

Q: 路网数据多大算正常?

- 统计节点、边和连通分量
- 规模异常时回查筛选与拆分流程

Q: 怎么验证构建的路网是对的?

- 比较节点、边和连通率
- 用QGIS检查形状
- 运行路径测试
- 随机抽样边属性

Q: Python处理太慢了怎么办?

- 先减少数据范围完成调试
- 优先使用向量化操作
- 必要时优化核心循环
- 再运行全量数据

第1次作业优秀示例与参考实现——可以参考的开源项目

优秀作业特征

- 模块划分清楚，函数有注释和测试
- 节点合并使用空间索引
- 边拆分覆盖所有必要拆分点
- 拓扑包含出入边和转向限制
- 质量报告包含指标和可视化
- README说明格式、用法和假设
- 输出数据可供G3和G4组直接使用

参考开源项目

- osmnx：理解OSM路网构建流程
- NetworkX：学习邻接结构和图算法
- pandana：参考城市网络分析
- OSRM：学习工业级数据结构
- GraphHopper：参考Java工程实现
- 建议先看整体流程，再学习算法与工程优化

实习1详细指导——第4周后的第一次进度检查

1 G2组汇报内容

- 展示数据探索和质量问题
- 说明节点合并与边拆分方法
- 展示拓扑、转向限制和质量评估
- 总结遇到的问题和解决方法

2 评分维度

- 功能完整性与代码质量占主要比重
- 可视化、文档和AI使用日志单独评分
- 答辩与演示按课程评分表执行

3 准备建议

- 提前准备10页以内的汇报PPT
- 重点展示前后对比和路网质量指标
- 现场演示代码或结果截图

实习1汇报PPT模板与要求——如何做一个清晰的进度汇报



第1页 概览

组号/成员/分工/当前进度(完成百分比)。一句话总结：我们做了什么、做到什么程度、遇到什么问题



第2-3页 数据探索

数据统计(节点/边/各等级道路里程)、发现的质量问题(截图+分析)、与预期的差异



第4-5页 核心算法

节点合并算法(原理/流程图/复杂度)、边拆分逻辑、拓扑构建方法。重点讲设计决策和选择理由



第6-7页 结果与质量

路网统计(节点/边/连通率)、质量评估(属性完整率/可视化对比)、与参考的对比(如OSM vs 构建后)

实习1评分标准与反馈机制——第一次进度检查



20%

数据探索

统计准确、问题分析有依据，并用可视化支持结论。



30%

算法实现

节点合并方法(空间索引加分)、边拆分正确性、拓扑完整性。有设计思考为优秀



25%

数据质量

报告连通性、属性完整率和验证过程；阈值按课程验收标准执行。



25%

汇报+代码

代码质量(模块化/注释)、文档完整性、汇报清晰度、回答问题。PPT专业+逻辑清晰为优秀

分组与分工建议——5人小组如何高效协作

组长(1人)	整体架构设计、进度管理、接口协调(与G1/G3/G4)、质量把控、汇报主讲。需要全局视野和沟通能力
数据处理工程师(2人)	数据读取与筛选、节点合并算法、边拆分与属性计算。核心算法开发，需要算法和编程能力强
拓扑与质量工程师(2人)	拓扑构建(邻接表/转向限制)、质量检查(连通率/可视化/验证)、文档与输出格式。需要细心和质量意识
协作方式	1) 每日站会(10分钟同步进度和问题)；2) Git分支开发(每人一个功能分支，合并到main)；3) 接口先行(先定义数据格式再开发)；4) 代码review(互相review保证质量)
工具推荐	Git(版本控制)、GitHub/Gitee(代码托管)、飞书/钉钉(沟通)、Trello/看板(任务管理)。工具是手段，高效协作才是目的
常见问题	分工不均(有人太忙有人太闲)、接口不对齐(各做各的最后接不上)、进度延期(低估难度)。解决方案：尽早沟通+接口先行+迭代开发

AI辅助开发规范与使用日志要求——如何正确使用AI工具

1 鼓励使用但必须理解

- AI可以辅助生成框架、解释算法和调试
- 提交者必须理解代码原理与实现细节

2 使用日志要求

- 记录时间、工具和使用目的
- 说明采纳内容及人工修改
- 将日志作为作业附件提交

3 注意事项

- 验证AI生成代码并补充测试
- 遇到陌生函数或API时查阅文档
- 逐行理解并修改代码
- 理解原理比复制结果更重要

CHAPTER · 核心问题

第3次课结束——路网是导航的骨架，下周空间索引见

从原始数据到可导航网络的6步基础构建与转向限制建模

6步基础构建(读取、筛选、合并、拆分、权重、拓扑)，再处理转向限制